Image_Page

เล่มที่ 2 · Frontend (หน้าเว็บ)

ผู้รับผิดชอบ: วิทย์

ฉบับวันที่ 13 ตุลาคม 2569

1. หน้าที่ของ Frontend

Frontend คือทุกอย่างที่ผู้ใช้และเจ้าหน้าที่เห็นในเบราว์เซอร์ รวมถึงตัวกลาง `proxy.py` ที่เสิร์ฟหน้าเว็บและส่งต่อคำขอ `/api/*` ไปให้ backend หน้าเว็บไม่คุยกับเครื่อง AI ตรง ๆ เลย ทุกอย่างผ่าน backend

- **Image_index:** หน้าเว็บหลักของผู้ใช้ มี 5 แท็บเครื่องมือ, ระบบบัญชี, ตั้งค่าภาษาและโน้ต
- **admin_panel:** Staff Console สำหรับเจ้าหน้าที่ ดูงานที่กำลังทำและจัดการผู้ใช้
- **proxy.py:** เว็บเซิร์ฟเวอร์ + reverse proxy ตัวเดียวที่ใช้กับทั้งสองหน้า

2. โครงสร้างไฟล์

ไฟล์	บรรทัด	หน้าที่
Image_index/index.html	168	โครงหน้า: แถบแท็บ, ฟอร์ม, พื้นที่ผลลัพธ์, หน้าต่างล็อกอิน / ตั้งค่า / จัดการแท็บ
Image_index/js/config.js	522	ค่าทั้งหมดที่เดียว: endpoint, ตัวเลือกของแต่ละแท็บ, ข้อความไทย/อังกฤษ, timeout
Image_index/js/api.js	677	ตัวเดียวที่คุยกับ backend: token, request, งาน (jobs + SSE), ผู้ใช้, admin
Image_index/js/eta.js	192	แถบเปอร์เซ็นต์และเวลาที่เหลือบนปุ่มสีส้ม
Image_index/js/script.js	776	ส่วนติดต่อผู้ใช้: แท็บ, อัปโหลด, ส่งงาน, แสดงผล, ล็อกอิน, ตั้งค่า
Image_index/css/style.css	1181	หน้าตาทั้งหมด รวมถึงแถบ % บนปุ่ม
admin_panel/index.html, js/admin.js, css/admin.css	115 / 299 / 342	Staff Console: แดชบอร์ดงาน + ตารางผู้ใช้
proxy.py	100	เสิร์ฟไฟล์หน้าเว็บ + ส่งต่อ /api/* ไป backend

ลำดับโหลดสคริปต์ใน index.html สำคัญ: **config.js → api.js → eta.js → script.js** เพราะ eta.js ต้องครอบฟังก์ชันของ api.js ก่อนที่ script.js จะเรียกใช้

> กฎของทีม: `script.js` ห้ามเรียก `fetch()` หรือแตะข้อมูลใน localStorage เอง ทุกการเชื่อมต่อต้องผ่าน `Api.*` ใน api.js ที่เดียว

3. config.js — ค่าตั้งของทั้งหน้า

แก้ตัวเลือกของแท็บ ข้อความ หรือ endpoint ที่ไฟล์นี้ไฟล์เดียว ไม่ต้องไปแก้ใน script.js

ค่า	ตอนนี้	ความหมาย
PUBLIC_MODE	true	โหมดใช้งานจริง: ต่อ backend จริงเสมอ ซ่อนป้ายและบัญชีทดสอบของโหมดจำลอง
API_MODE	"live"	live = backend จริง · mock = ข้อมูลจำลองในเบราว์เซอร์ (ไว้พัฒนา UI) · auto = ลอง backend ก่อน
API_BASE	""	เรียก /api/... ที่เดียวกับหน้าเว็บ (ผ่าน proxy.py)
REQUEST_TIMEOUT	120 วินาที	เวลารอสูงสุดของคำขอทั่วไป
PROCESS_TIMEOUT	330 วินาที	เวลารองานประมวลผลภาพ
HEALTH_INTERVAL	30 วินาที	ถาม /api/v1/health เพื่ออัปเดตป้ายสถานะการเชื่อมต่อ
NET_RETRY	1	ลองซ้ำเมื่อเน็ตสะดุด (เฉพาะ GET)
STORE	"imgstudio:"	คำนำหน้าของทุกคีย์ใน localStorage
TAB_ORDER / TAB_CONFIG	5 แท็บ	ชื่อแท็บ, endpoint, ชนิดข้อมูลที่ส่ง, ตัวเลือก (enum / range / bool) และค่าเริ่มต้น
I18N	th, en	ข้อความทุกจุดบนหน้าเว็บทั้งสองภาษา

ตัวเลือกของแต่ละแท็บ (TAB_CONFIG)

แท็บ	endpoint	ส่งเป็น	ฟิลด์
genimage	/api/v1/generate	JSON	prompt, negative_prompt, lora (none, pixel_art, linear_manga, outline), size (ค่าเริ่มต้น 512x768)
back	/api/v1/remove-background	multipart	image, output (transparent / white / black), refine_edge
icon (Adjust)	/api/v1/adjust	multipart	image, brightness, contrast, saturation, temperature, vignette
tone	/api/v1/filter	multipart	image, filter_name (10 แบบ ค่าเริ่มต้น warm), intensity 0–100 (ค่าเริ่มต้น 70)
blur	/api/v1/blur	multipart	image, blur_type (gaussian / background / face), blur_amount 0–100 (ค่าเริ่มต้น 40)

แท็บ Adjust ยังใช้ชื่อภายในว่า icon เพราะเดิมเป็นแท็บลบลายน้ำ ผู้ใช้เห็นชื่อ Adjust

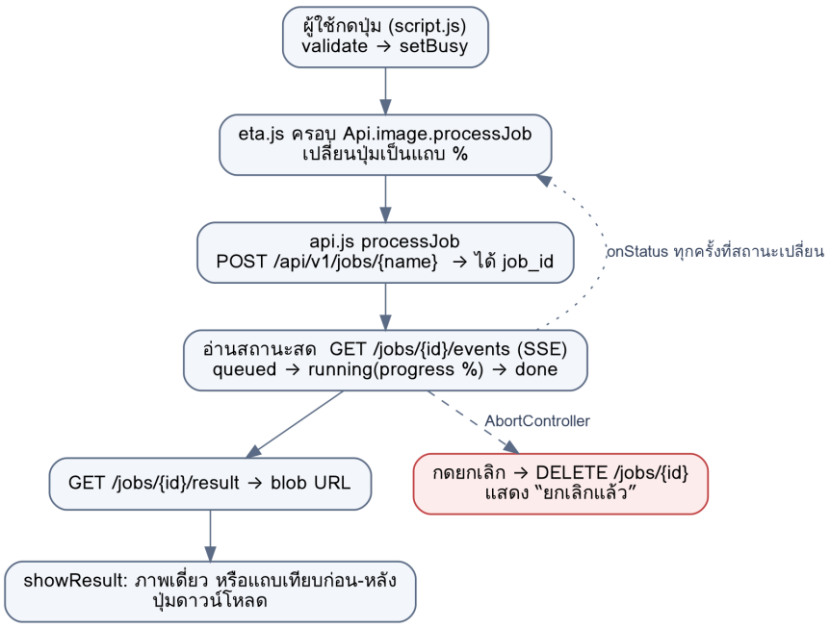
4. api.js — ตัวกลางคุยกับ backend

4.1 ส่วนประกอบ

- **Token / Session:** เก็บ token และข้อมูลผู้ใช้ใน localStorage (คีย์ขึ้นต้น `imgstudio:`) ถ้า backend ตอบ 401 จะล้าง session แล้วเปิดหน้าต่างล็อกอิน
- **request():** ตัวเรียก API ตัวเดียว ใส่ `Authorization: Bearer <token>`, มี timeout, ลองซ้ำ GET 1 ครั้ง, แปลง error เป็น `ApiError` ที่มี code และข้อความภาษาไทย
- **ป้ายสถานะ:** ถาม /api/v1/health ทุก 30 วินาที แล้วแจ้ง script.js ผ่าน `Api.onStatus`
- **Api.auth:** register, login, logout, me
- **Api.user:** อ่าน/บันทึกโน้ต และ preferences (ภาษา + แท็บที่แสดง)
- **Api.image.processJob:** ส่งงานภาพผ่านระบบคิว (หัวข้อ 4.2)
- **Api.admin:** รายชื่อผู้ใช้, เปลี่ยนสิทธิ์, ลบ (ใช้ในหน้าหลักเมื่อผู้ใช้เป็น staff)

4.2 การส่งงานภาพ (processJob)

1. สร้าง body ตามชนิดของแท็บ (JSON สำหรับ GenImage, multipart สำหรับแท็บที่มีรูป)
2. `POST /api/v1/jobs/{name}` → ได้ `job_id` และสถานะแรก (queued)
3. เปิด `GET /api/v1/jobs/{id}/events` (SSE) อ่านสถานะสดทีละก้อน แล้วเรียก `onStatus(สถานะ)` ทุกครั้ง
4. เมื่อสถานะเป็น `done` → `GET /api/v1/jobs/{id}/result` ได้ภาพเป็น blob แล้วคืนเป็น blob URL
5. ถ้าเป็น `error` โยน ApiError ตามรหัสที่ backend ส่งมา · ถ้าผู้ใช้กดยกเลิก (AbortController) ส่ง `DELETE /api/v1/jobs/{id}` ให้ backend ยกเลิกงานด้วย



ภาพที่ 1 · ตั้งแต่กดปุ่มจนได้ภาพ

5. eta.js — แถบเปอร์เซ็นต์บนปุ่ม

eta.js ครอบ `Api.image.processJob` ไว้ ทุกครั้งที่ backend ส่งสถานะมา
ปุ่มสีส้มจะเปลี่ยนข้อความและความยาวแถบตามนั้น แล้วคืนปุ่มเป็นข้อความเดิมเมื่องานจบหรือถูกยกเลิก

สถานะ	ปุ่มแสดง	ที่มาของตัวเลข
ส่งคำขอ	กำลังส่ง... + แถบวิ่งไปมา	—
queued	คิวที่ 2 จาก 3 · รอมา 0:15 + แถบวิ่งไปมา	position / total จาก backend
running (สร้างภาพ)	45% · เหลือ ~6 วิ	progress และ eta_sec จริงจาก Forge
running (อีก 4 แท็บ)	~60% · เหลือ ~3 วิ	ประมาณจาก avg_sec ของ backend หรือสถิติในเครื่อง (5 ครั้งล่าสุด)
done	100% แล้วคืนปุ่มเดิม	—

- แถบไม่ถอยหลัง และค้างที่ 99% จนกว่างานจะเสร็จจริง
- การประมาณจากเวลา: วิ่งตามเวลาจริงถึง 90% จากนั้นชะลอเข้าหา 99%
- ค่าเริ่มต้นเมื่อยังไม่มีสถิติ: GenImage 90 วิ, Back 15 วิ, Adjust 5 วิ, Tone 5 วิ, Blur 8 วิ
- ข้อความแปลตามภาษาที่เลือก (คีย์ `eta.*` ใน config.js)

6. script.js — ส่วนติดต่อผู้ใช้

ไฟล์แบ่งเป็น 10 ส่วนตามหัวคอมเมนต์ในโค้ด

ส่วน	ทำอะไร
0. Blob Registry	จัดการ blob URL ของภาพ (1 URL มีเจ้าของเดียว) กันหน่วยความจำรั่ว
0.5 ป้ายสถานะ	ป้ายเชื่อมต่อ / ไม่เชื่อมต่อ ที่มุมบน
1. Tab bar + จัดการแท็บ	สร้างแถบแท็บ, ปุ่มดินสอสำหรับซ่อนหรือเรียงแท็บ (ต้องเหลืออย่างน้อย 1 แท็บ)
2. switchTab + renderOptions	สลับแท็บ แล้วสร้างช่องตัวเลือกตาม TAB_CONFIG, ล็อกฟอร์มถ้ายังไม่ล็อกอิน
3. ไฟล์อัปโหลด	ลากวางหรือเลือกไฟล์, แสดงรูปย่อ, ชื่อ, ขนาด
4. ผลลัพธ์ / ดาวน์โหลด	แสดงภาพเดี่ยว หรือแถบเลื่อนเทียบก่อน-หลัง, ปุ่มดาวน์โหลด
5. Validate + Submit	ตรวจว่ามี prompt หรือไฟล์ แล้วส่งงานผ่าน Api.image.processJob, ปุ่มยกเลิก
6. ล็อกอิน / บัญชี	หน้าต่างสมัคร / ล็อกอิน, เมนูบัญชี, ปุ่มไป Staff Console เมื่อเป็น staff
7. ตั้งค่า	เลือกภาษา, โน้ตส่วนตัว (บันทึกอัตโนมัติหลังหยุดพิมพ์ 0.5 วิ), ปุ่มรีเซ็ตข้อมูลในเครื่อง
8. เมนู / Modal / URL	เมนู ☰, หน้าต่างต่าง ๆ, จำแท็บปัจจุบันไว้ใน URL (#tab)
9. Boot	เริ่มทำงานตอนเปิดหน้า: โหลดค่า, ตรวจ session, เปิดแท็บแรก

7. Staff Console (admin_panel)

หน้าแยกสำหรับเจ้าหน้าที่ เปิดผ่าน `python proxy.py admin` ที่ 127.0.0.1:8090 จึงเข้าได้เฉพาะจากเครื่อง frontend ล็อกอินด้วยบัญชีที่มีสิทธิ์ staff เท่านั้น token เก็บใน sessionStorage ปิดแท็บแล้วหลุดเอง

7.1 แดชบอร์ดงาน

- ดึง `GET /api/v1/admin/jobs` ทุก 2 วินาที (หยุดเองเมื่อแท็บถูกซ่อน ปิดอัตโนมัติได้)
- การ์ดสรุป 2 สาย: กำลังทำ x/1 และรอคิว y/10 การ์ดเป็นสีเหลืองเมื่อมีงาน และเป็นสีแดงเมื่อคิวเต็ม
- ตารางงานที่กำลังทำ: ชื่อ อีเมล IP, เครื่องมือ, สถานะ + แถบ %, เวลารอคิว, เวลาประมวลผล, ตัวเลือกที่ส่งมา (เช่น prompt)
- ส่วนพับได้: งานที่จบใน 10 นาทีล่าสุด พร้อมผลสำเร็จ / ผิดพลาดและสาเหตุ

7.2 จัดการผู้ใช้

- ตารางผู้ใช้: ชื่อ, อีเมล, สิทธิ์, วันสมัคร
- เปลี่ยนสิทธิ์ user ↔ staff จาก dropdown และลบบัญชี (มีหน้าต่างยืนยัน)
- บัญชีของตัวเองถูกล็อก ลดสิทธิ์หรือลบตัวเองไม่ได้

8. proxy.py

หน้าที่	รายละเอียด
เสิร์ฟหน้าเว็บ	`python proxy.py user` → Image_index ที่ 0.0.0.0:8080 · `python proxy.py admin` → admin_panel ที่ 127.0.0.1:8090
ส่งต่อ /api/*	ทุก method (GET, POST, PUT, PATCH, DELETE) ไปที่ BACKEND_URL แบบสตรีม จึงใช้กับ SSE ได้
จำกัดขนาด	เกิน 25 MB ตอบ 413 FILE_TOO_LARGE ทันที ไม่ส่งต่อ (นับเองแม้ไม่มี Content-Length)
IP ผู้ใช้	ทิ้ง X-Forwarded-For / X-Real-IP ที่ผู้ใช้ส่งมา แล้วใส่ IP จริงเอง
backend ล่ม / ช้า	ต่อไม่ติด → 502 BACKEND_DOWN · ช้าเกิน → 504 TIMEOUT

```
# เครื่อง frontend (โฟลเดอร์ Image_Page)
set BACKEND_URL=http://26.23.191.235:5050
python proxy.py user      # หน้าเว็บผู้ใช้  http://<IP>:8080
python proxy.py admin     # Staff Console   http://127.0.0.1:8090
```

> ค่า BACKEND_URL เริ่มต้นในโค้ดคือ http://26.23.191.235:5050 ถ้า IP ของเครื่อง backend ใน Radmin เปลี่ยน ให้ตั้ง env ใหม่ ไม่ต้องแก้โค้ด

9. การทดสอบที่เกี่ยวกับ Frontend

ไฟล์เทส	สิ่งที่เช็ค
test_frontend_browser.py	เปิดหน้าเว็บจริงใน Chromium: สร้างภาพแล้วแถบ % ขยับไม่ถอยหลัง, กดยกเลิกแล้วปุ่มกลับปกติและงานใน backend ถูกยกเลิก, อัปโหลดรูปแท็บ Tone, ฟังก์ชัน Eta.percent
test_proxy_and_outputs.py	proxy.py ส่งต่อครบ, ตัดไฟล์ใหญ่, ไม่ส่ง IP ปลอม, backend ล่ม / ช้า, Staff Console เปิดได้เฉพาะ localhost
test_generate_options.py	ตัวเลือก LoRA และขนาดภาพใน config.js ตรงกับฝั่ง AI

ปัญหาที่รู้อยู่: ตอนคิวว่าง ปุ่มขึ้น "คิวที่ 0 จาก 1" แวบหนึ่งก่อนเริ่มงาน (เทส `test_button_never_shows_queue_position_zero` ไม่ผ่าน) ทีมตัดสินใจปล่อยไว้ ถ้าจะแก้ให้ eta.js แสดง "กำลังส่ง..." เมื่อ position น้อยกว่า 1